



CMPT 413/713: Natural Language Processing

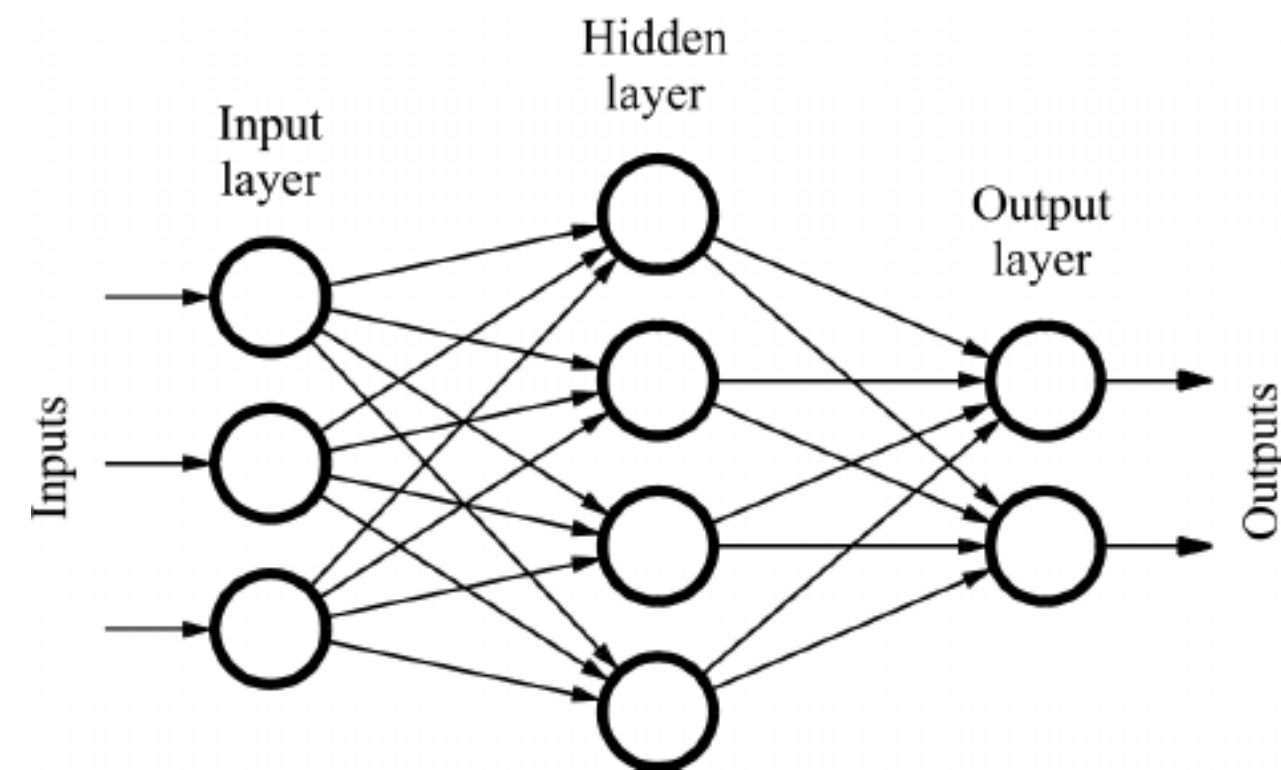
Neural Network Basics

Fall 2021
2021-09-23

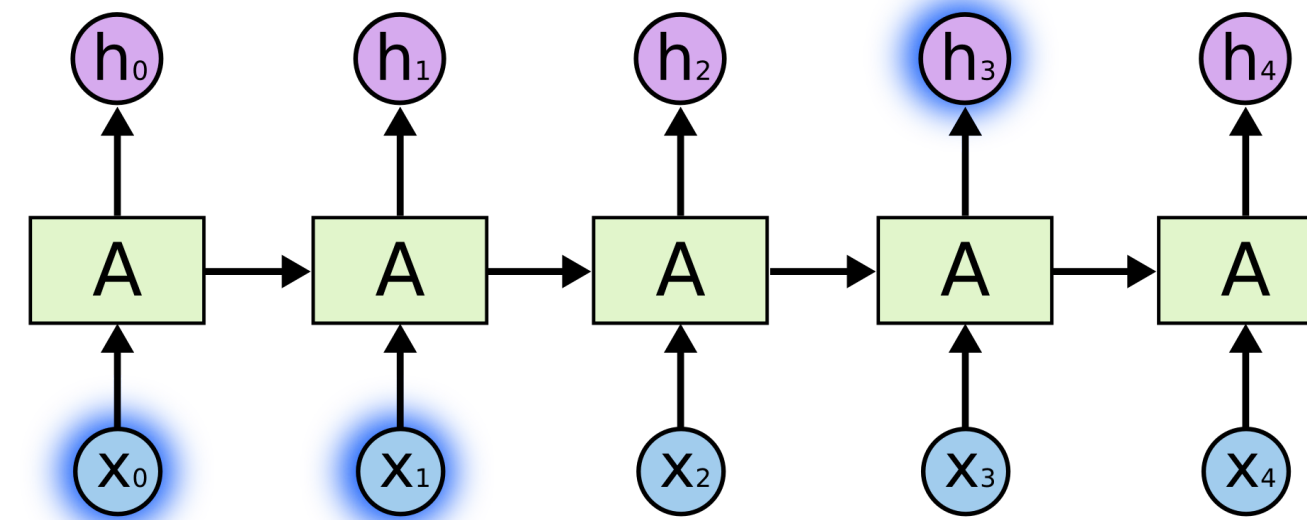
Adapted from slides from Danqi Chen and Karthik Narasimhan

Neural networks for NLP

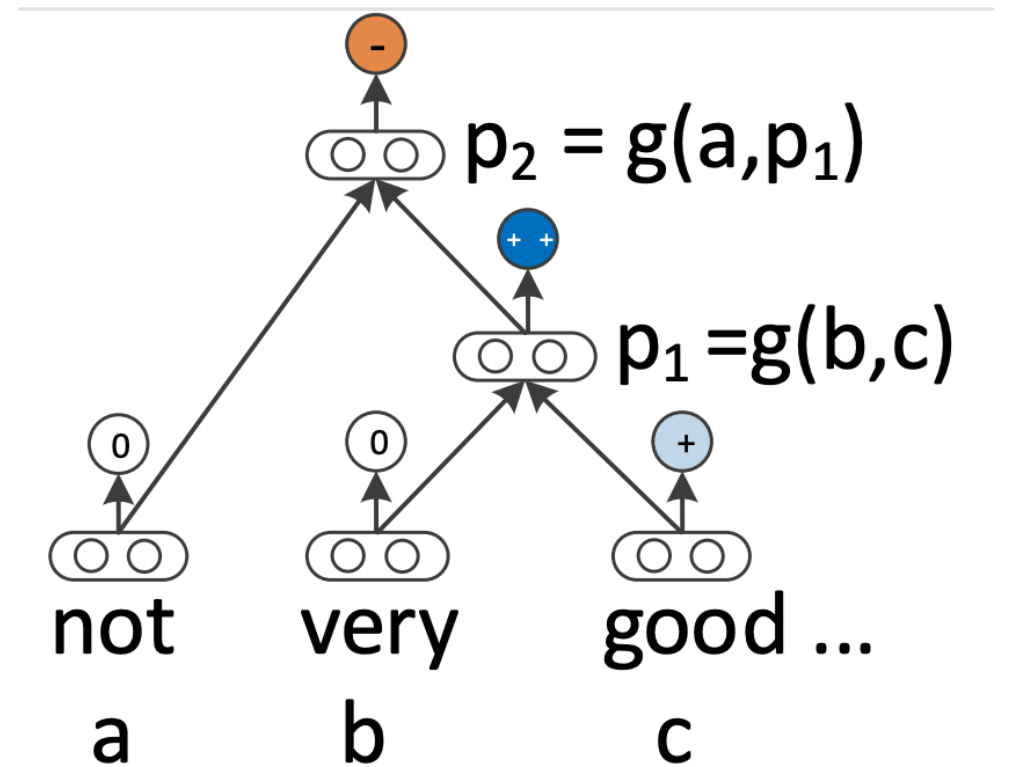
Feed-forward NNs



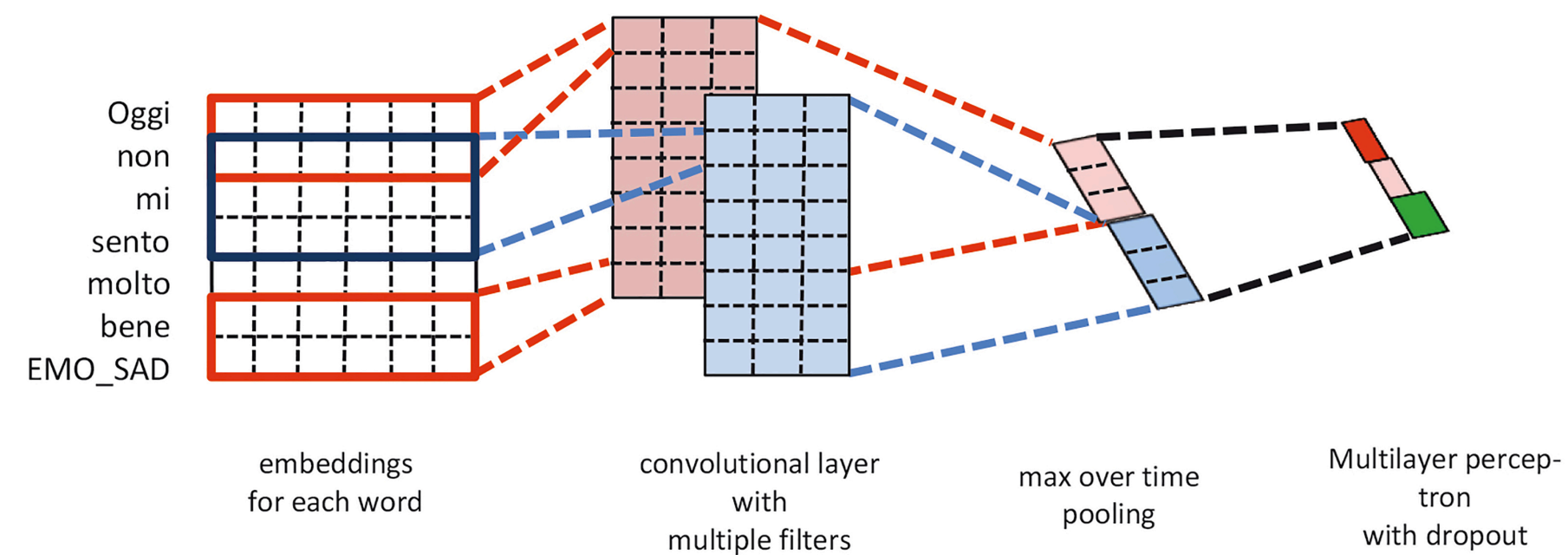
Recurrent NNs



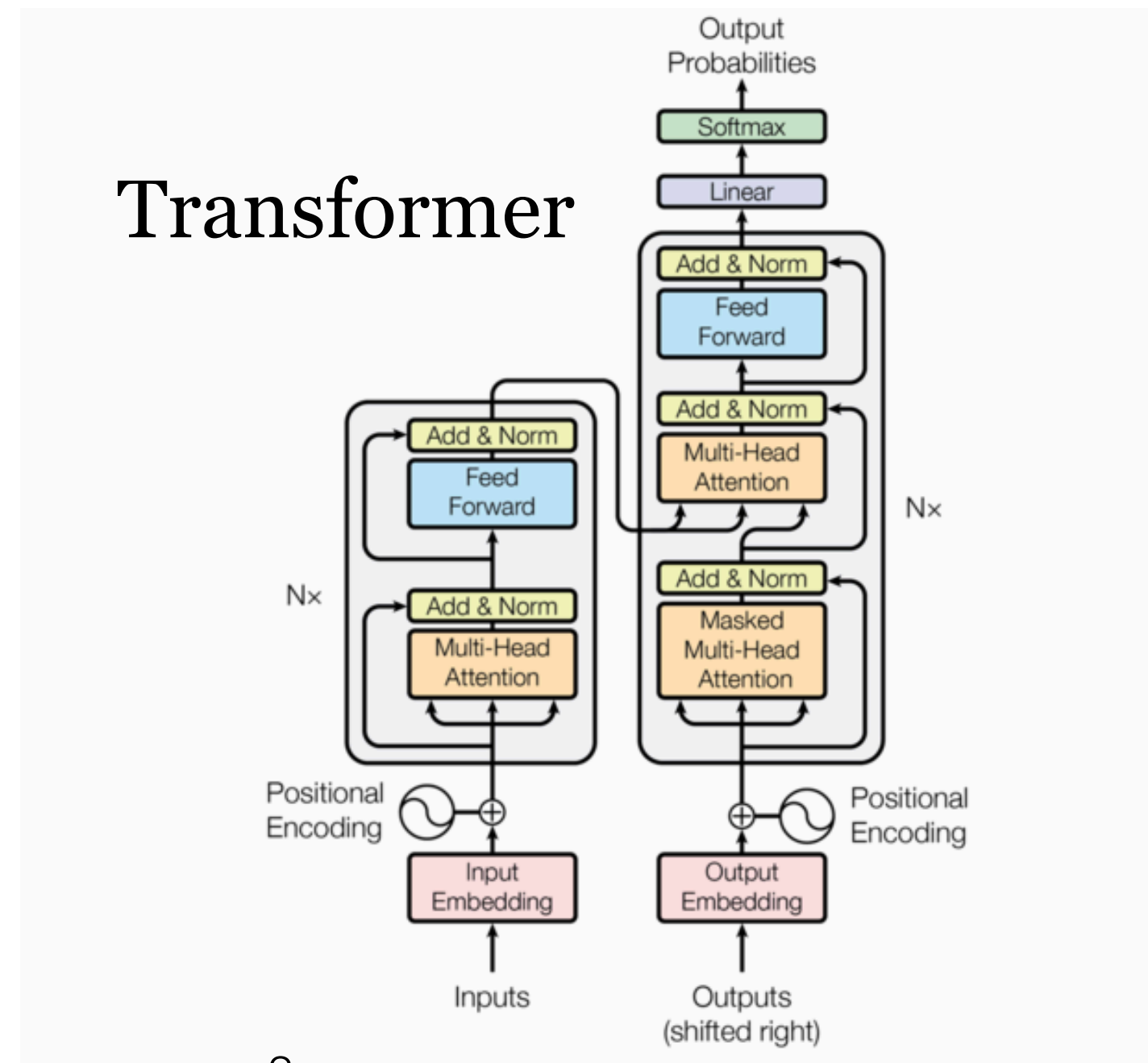
Recursive NNs



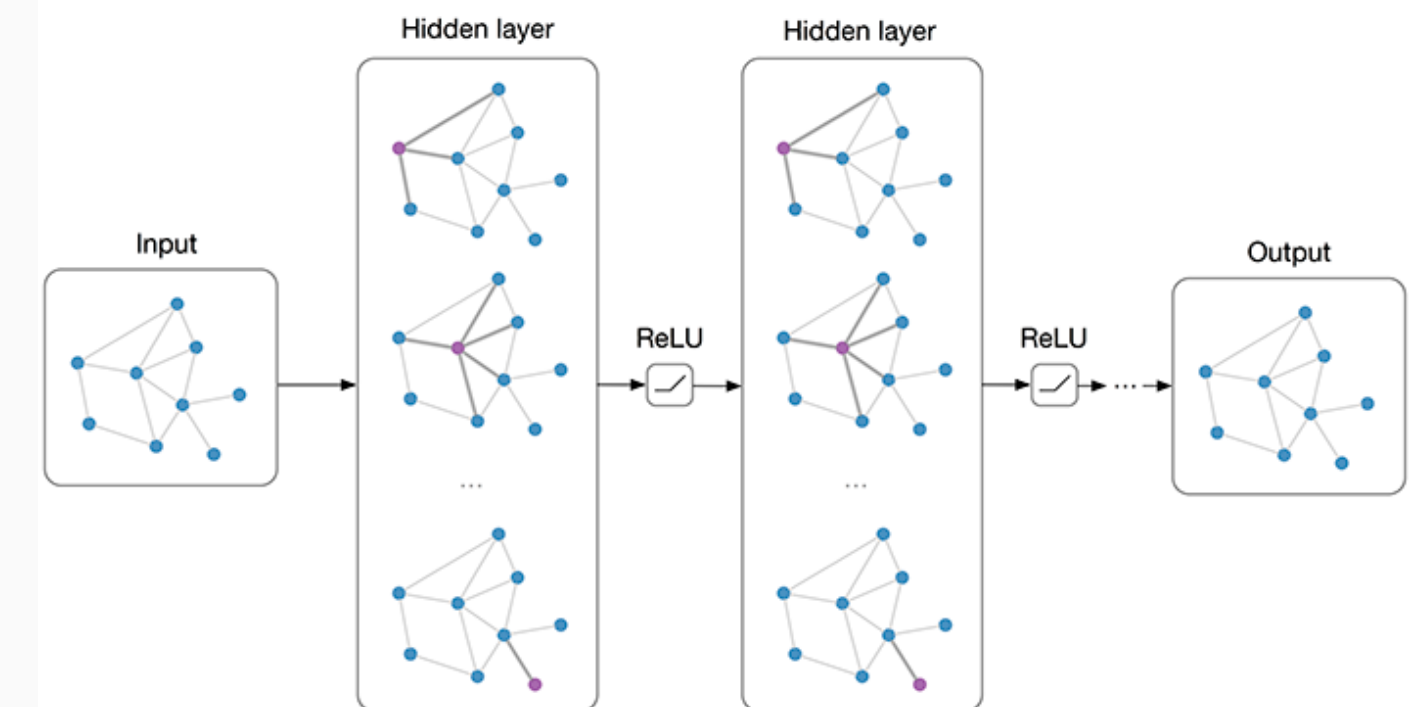
Convolutional NNs



Transformer



Graph NNs



Always coupled with word embeddings...

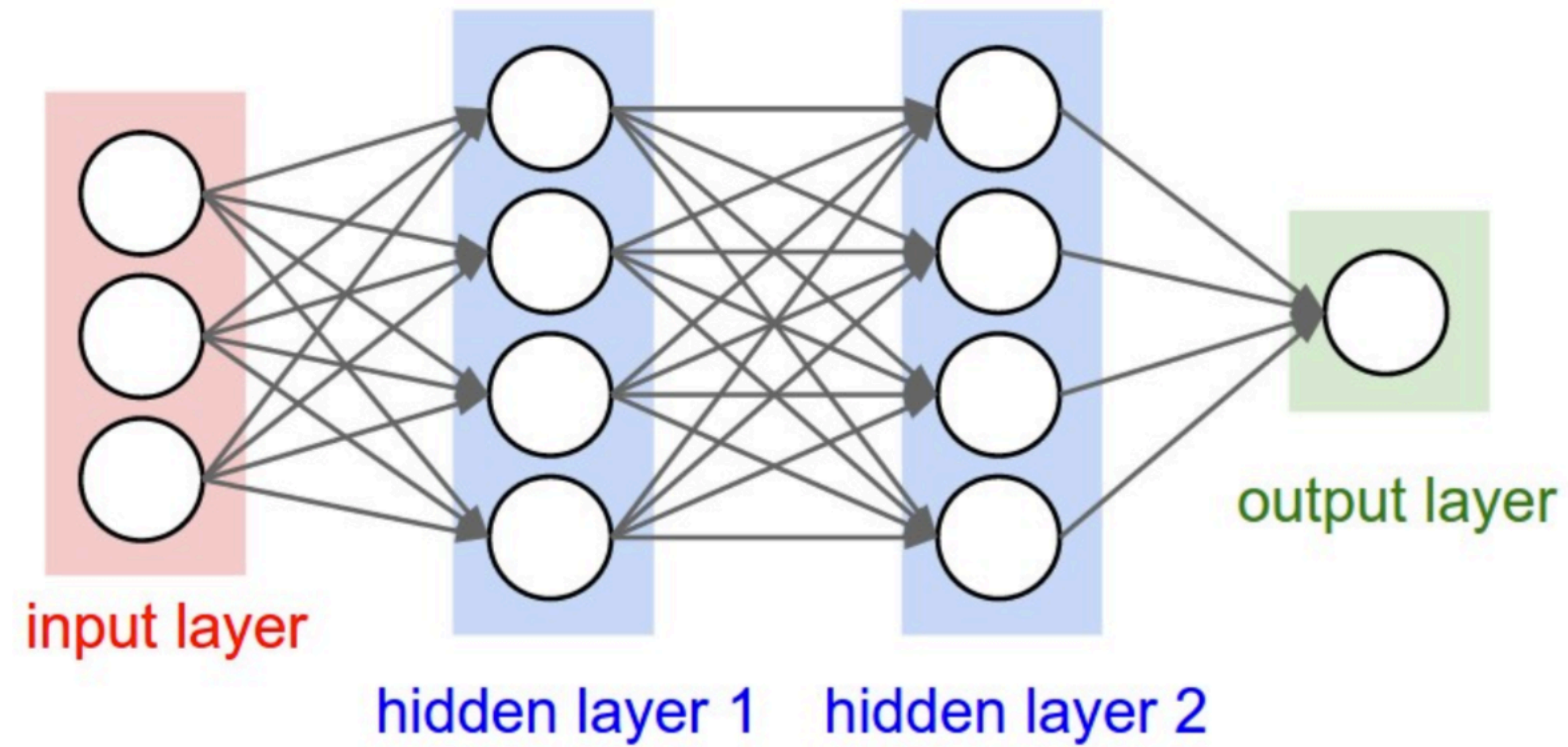
This Lecture

- Feedforward Neural Networks
 - Hidden layers
 - Activation function
- Applications
 - Neural Bag-of-Words Models
 - Feedforward Neural Language Models
- The training algorithm: Back-propagation

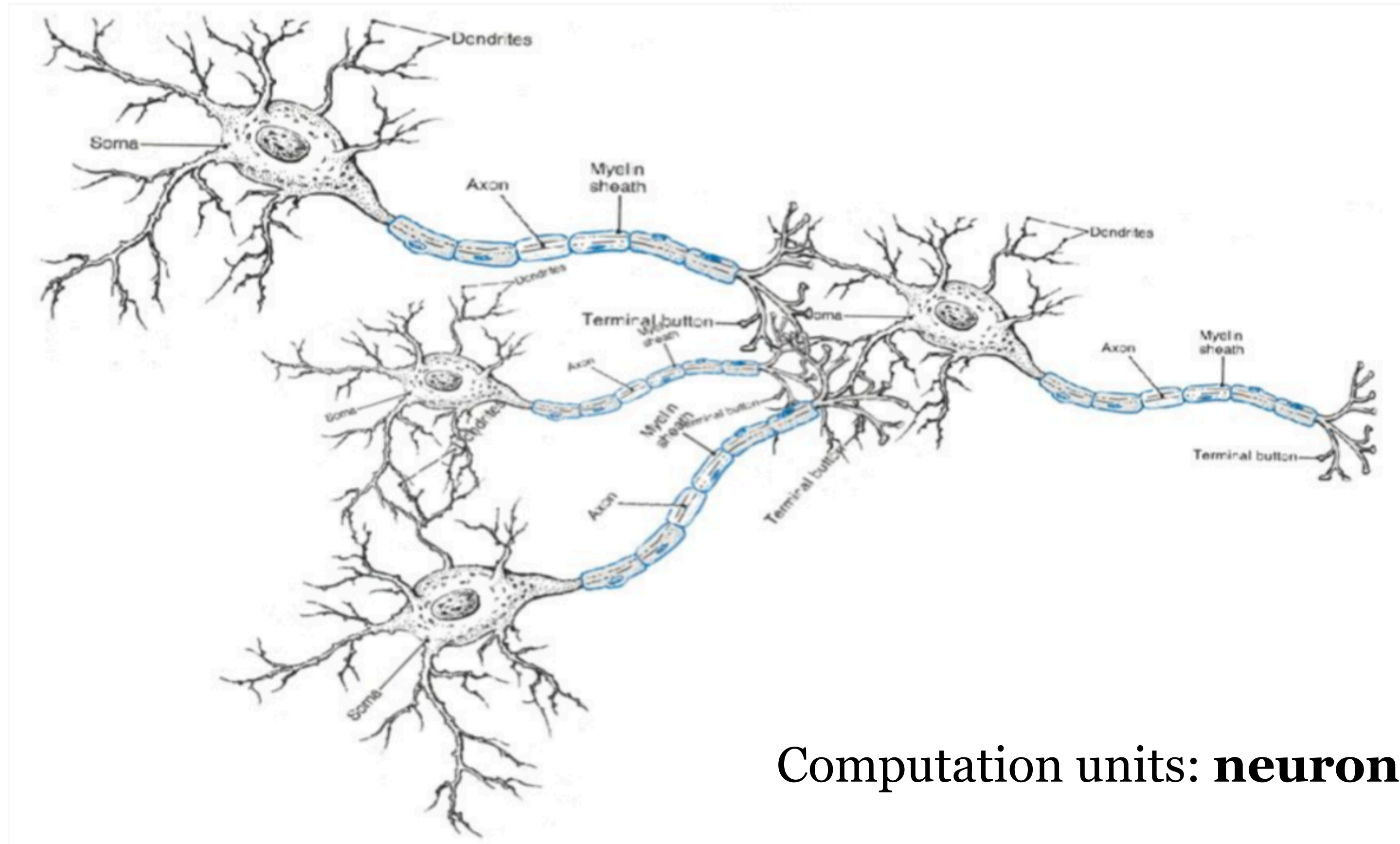
Feed-forward Neural Networks

Feed-forward NNs

- Input: $x_1, \dots, x_d$
- Output: $y \in \{0,1\}$



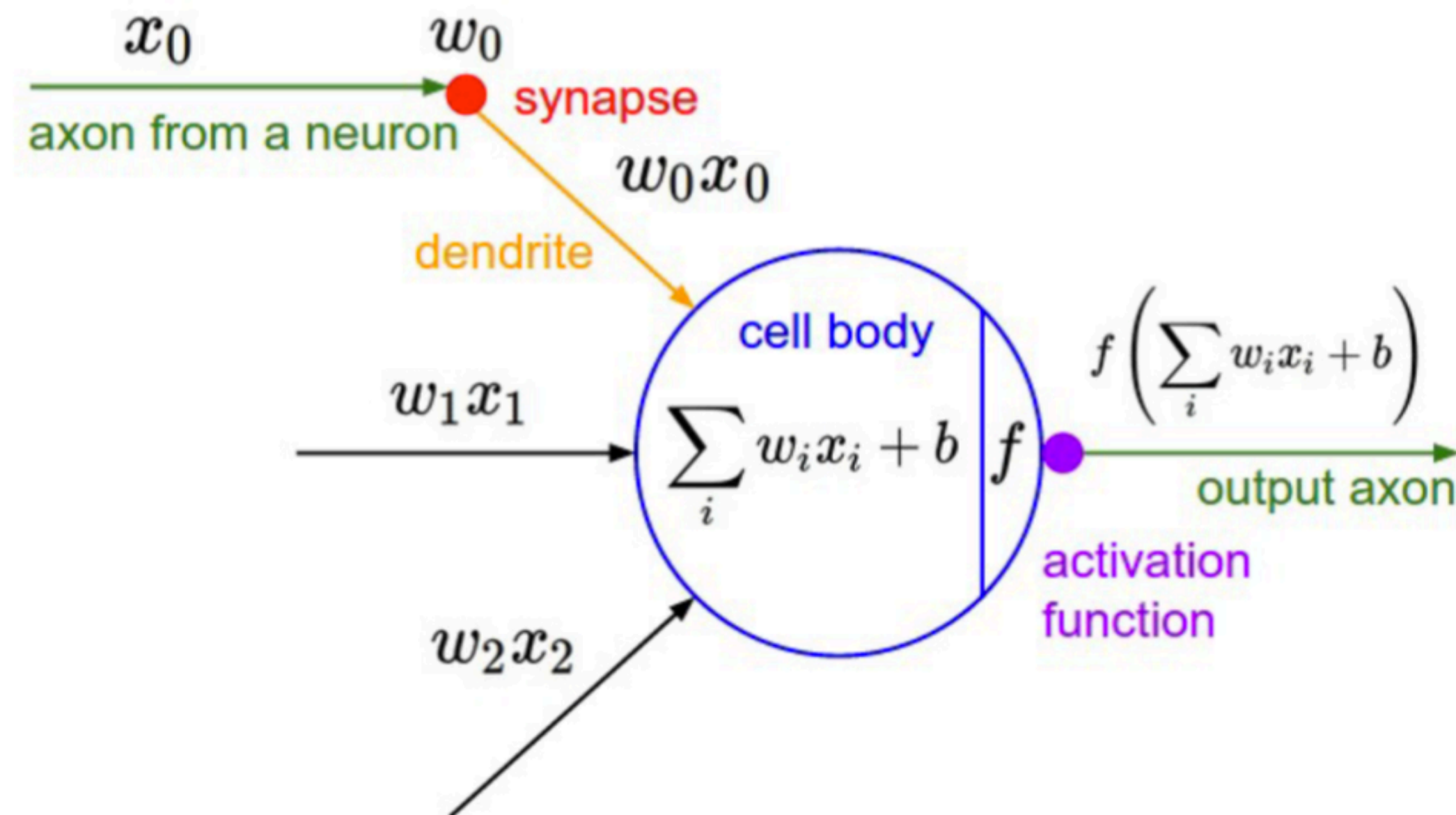
Neural computation



Computation units: **neurons**

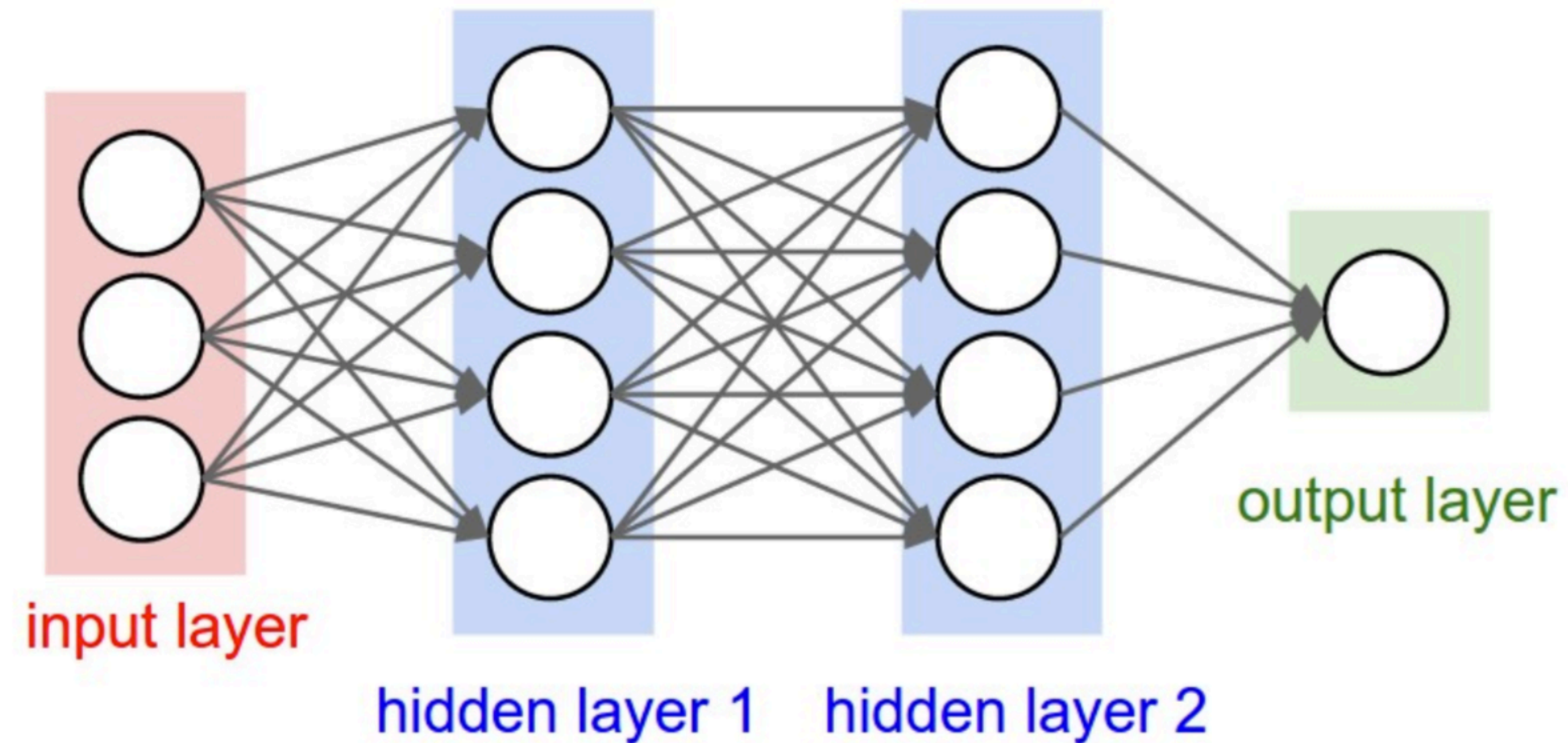
An artificial neuron

- A neuron is a computational unit that has scalar inputs and an output
- Each input has an associated weight.
- The neuron multiplies each input by its weight, sums them, applied a **nonlinear function** to the result, and passes it to its output.

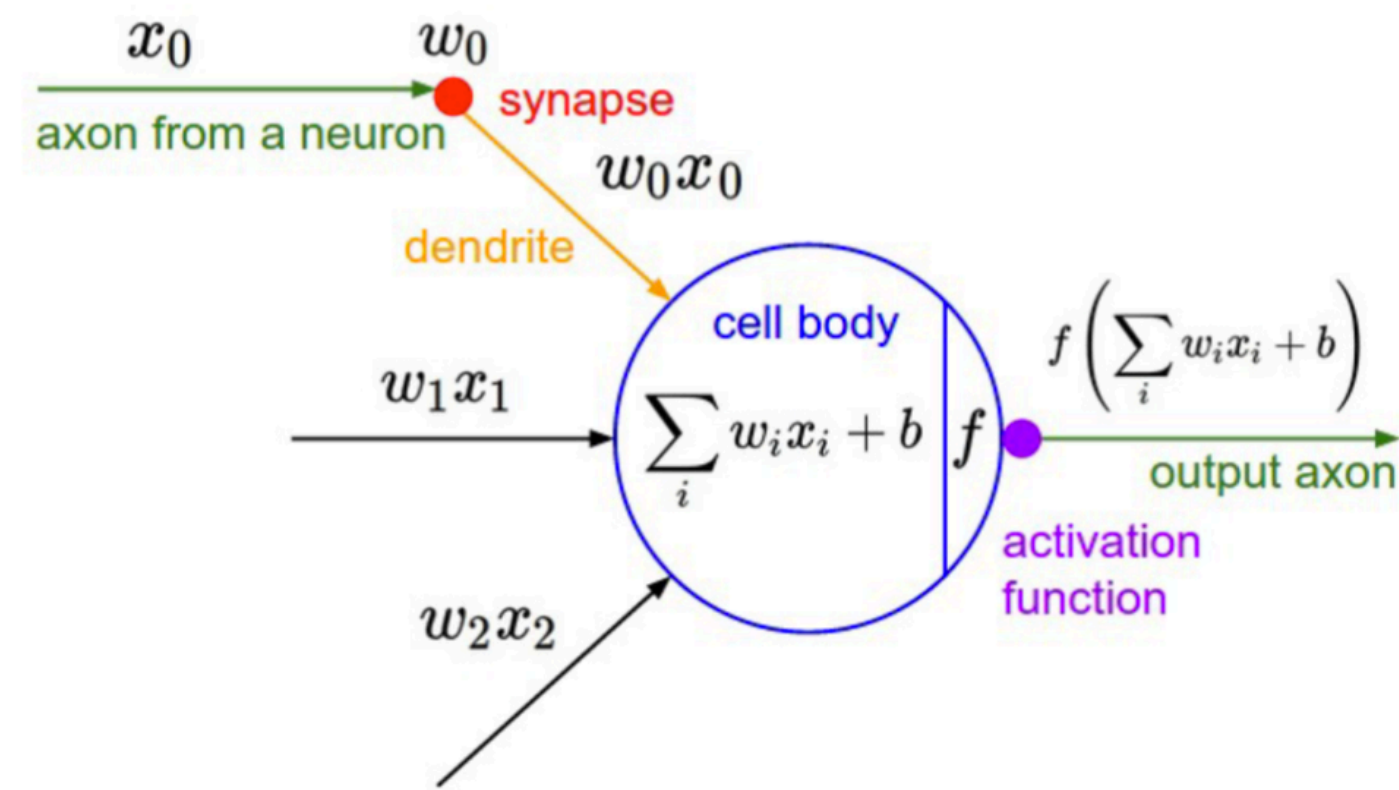


Neural networks

- The neurons are connected to each other, forming a **network**
- The output of a neuron may feed into the inputs of other neurons

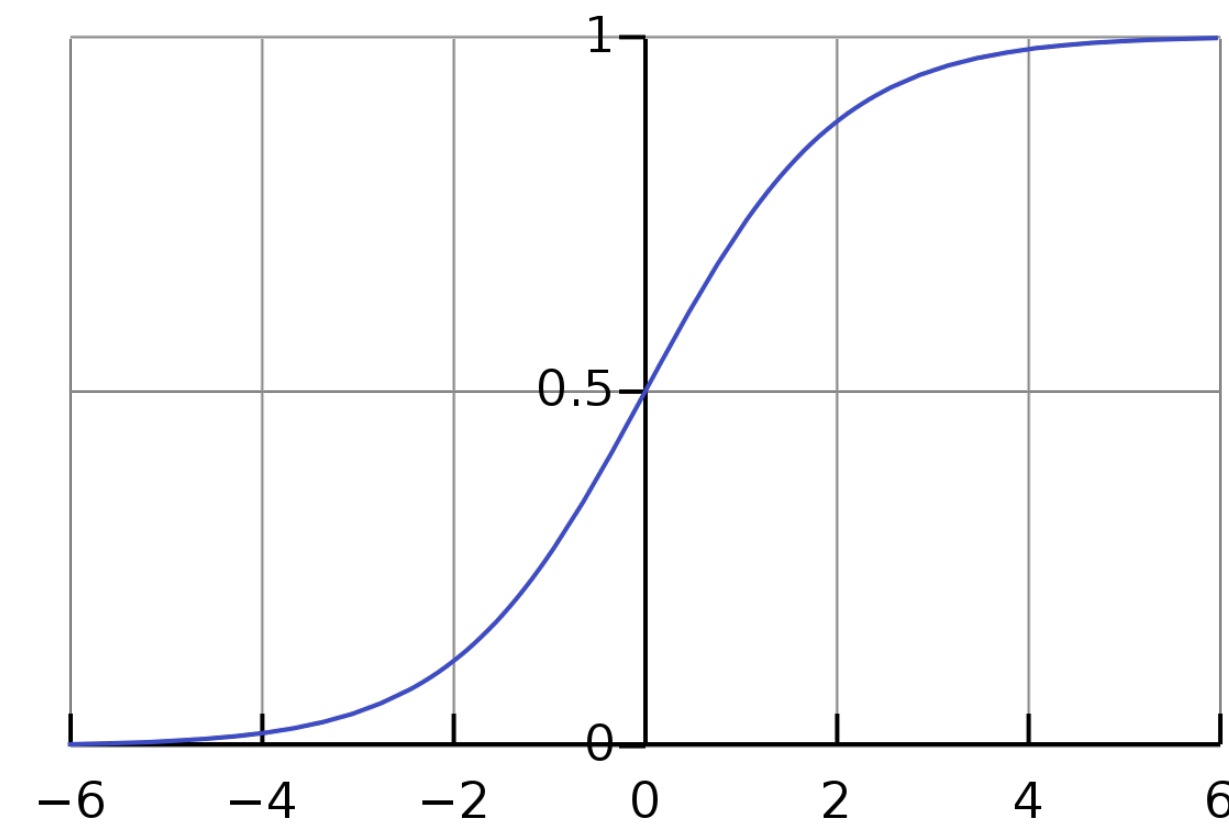


A neuron can be a binary logistic regression unit

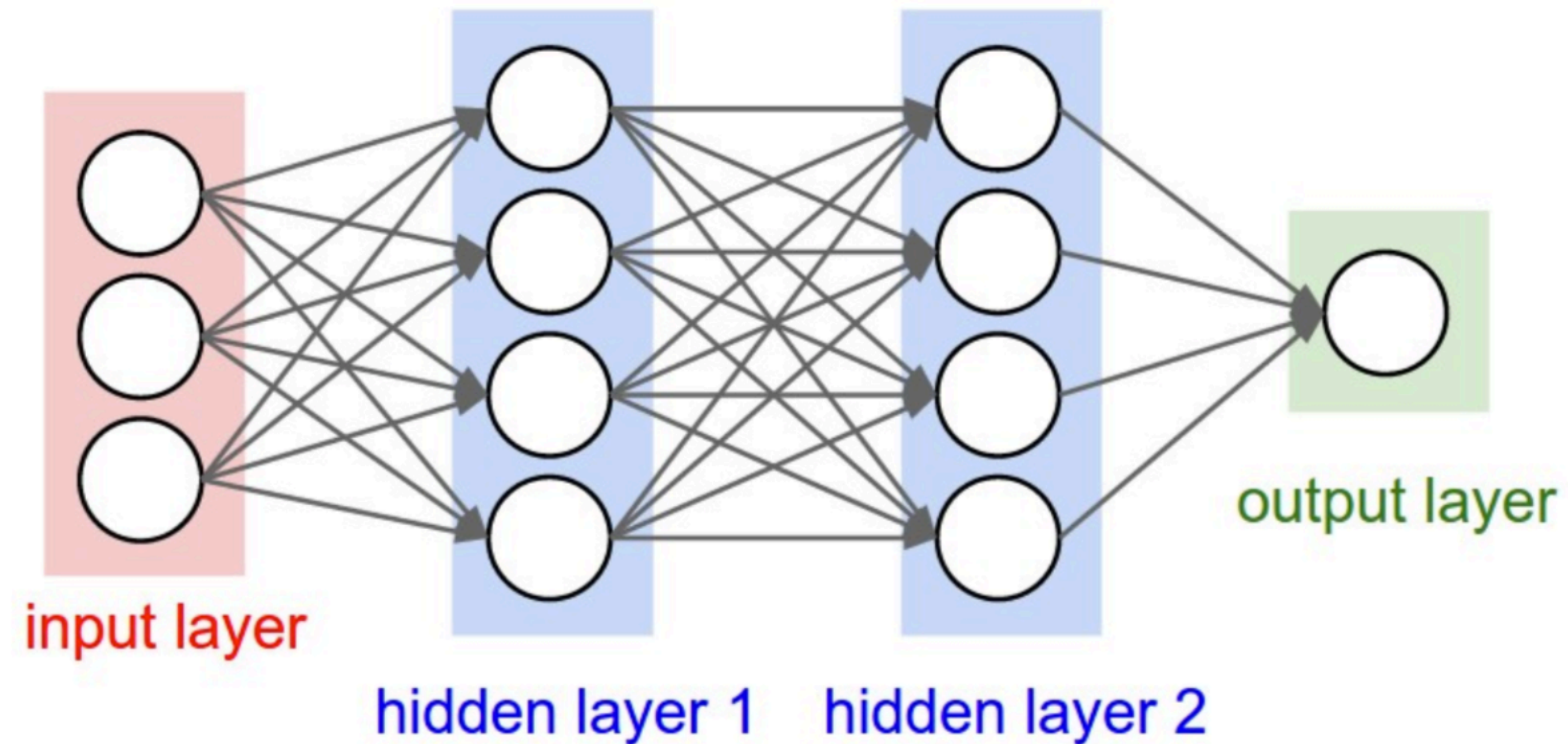


$$f(z) = \frac{1}{1 + e^{-z}}$$

$$h_{\mathbf{w},b}(\mathbf{x}) = f(\mathbf{w}^\top \mathbf{x} + b)$$



A neural network
= running several logistic regressions at the same time



- If we feed a vector of inputs through a bunch of logistic regression functions, then we get a vector of outputs...
- which we can feed into another logistic regression function

Mathematical Notations

- Input layer: $x_1, \dots, x_d$

- Hidden layer 1: $h_1^{(1)}, h_2^{(1)}, \dots, h_{d_1}^{(1)}$

$$h_1^{(1)} = f(W_{1,1}^{(1)}x_1 + W_{1,2}^{(1)}x_2 + \dots + W_{1,d}^{(1)}x_d + b_1^{(1)})$$

$$h_2^{(1)} = f(W_{2,1}^{(1)}x_1 + W_{2,2}^{(1)}x_2 + \dots + W_{2,d}^{(1)}x_d + b_2^{(1)})$$

...

- Hidden layer 2: $h_1^{(2)}, h_2^{(2)}, \dots, h_{d_2}^{(2)}$

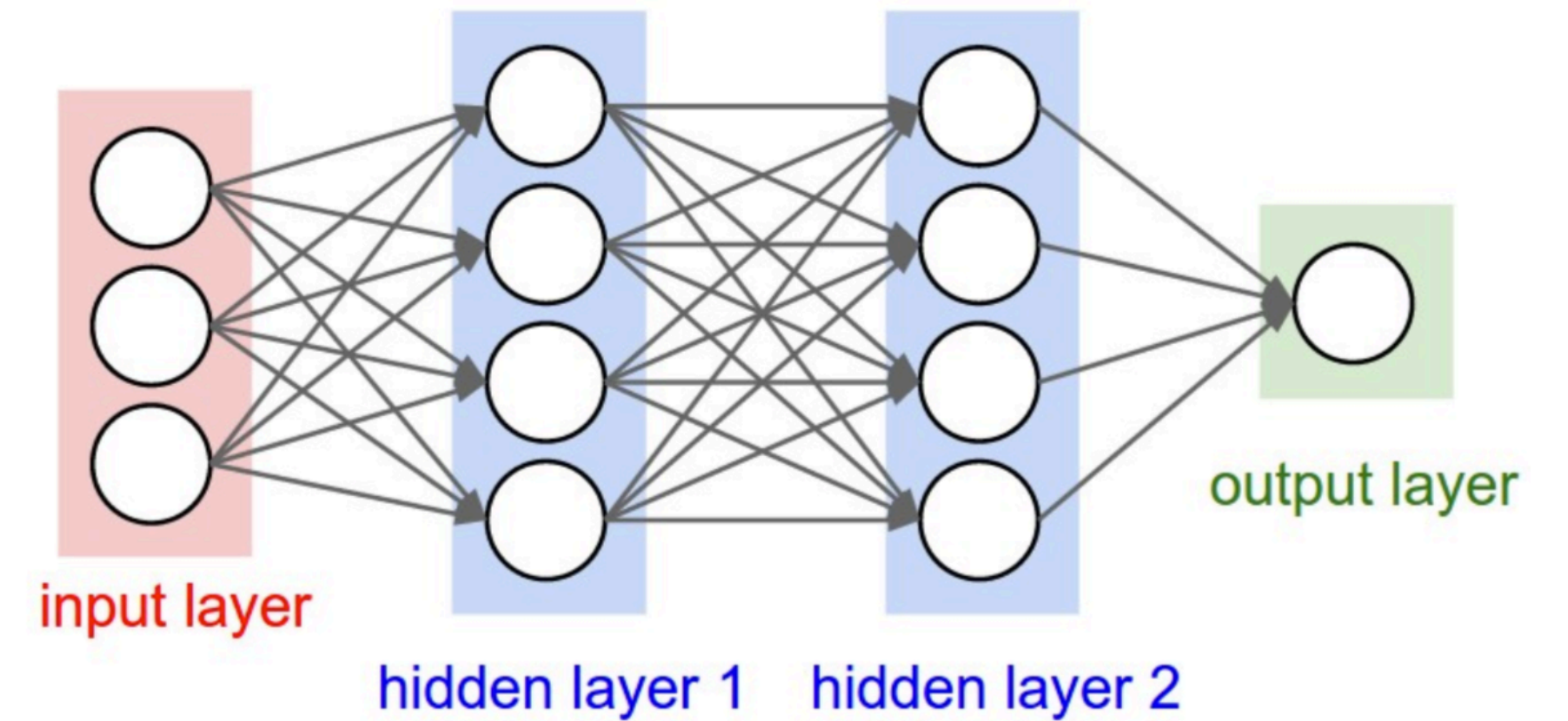
$$h_1^{(2)} = f(W_{1,1}^{(2)}h_1^{(1)} + W_{1,2}^{(2)}h_2^{(1)} + \dots + W_{1,d_1}^{(2)}h_{d_1}^{(1)} + b_1^{(2)})$$

$$h_2^{(2)} = f(W_{2,1}^{(2)}h_1^{(1)} + W_{2,2}^{(2)}h_2^{(1)} + \dots + W_{2,d_1}^{(2)}h_{d_1}^{(1)} + b_2^{(2)})$$

...

- Output layer:

$$y = \sigma(w_1^{(o)}h_1^{(2)} + w_2^{(o)}h_2^{(2)} + \dots + w_{d_2}^{(o)}h_{d_2}^{(2)} + b^{(o)})$$



Matrix Notations

- Input layer: $\mathbf{x} \in \mathbb{R}^d$

- Hidden layer 1:

$$\mathbf{h}_1 = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \in \mathbb{R}^{d_1}$$

$$\mathbf{W}^{(1)} \in \mathbb{R}^{d_1 \times d}, \mathbf{b}^{(1)} \in \mathbb{R}^{d_1}$$

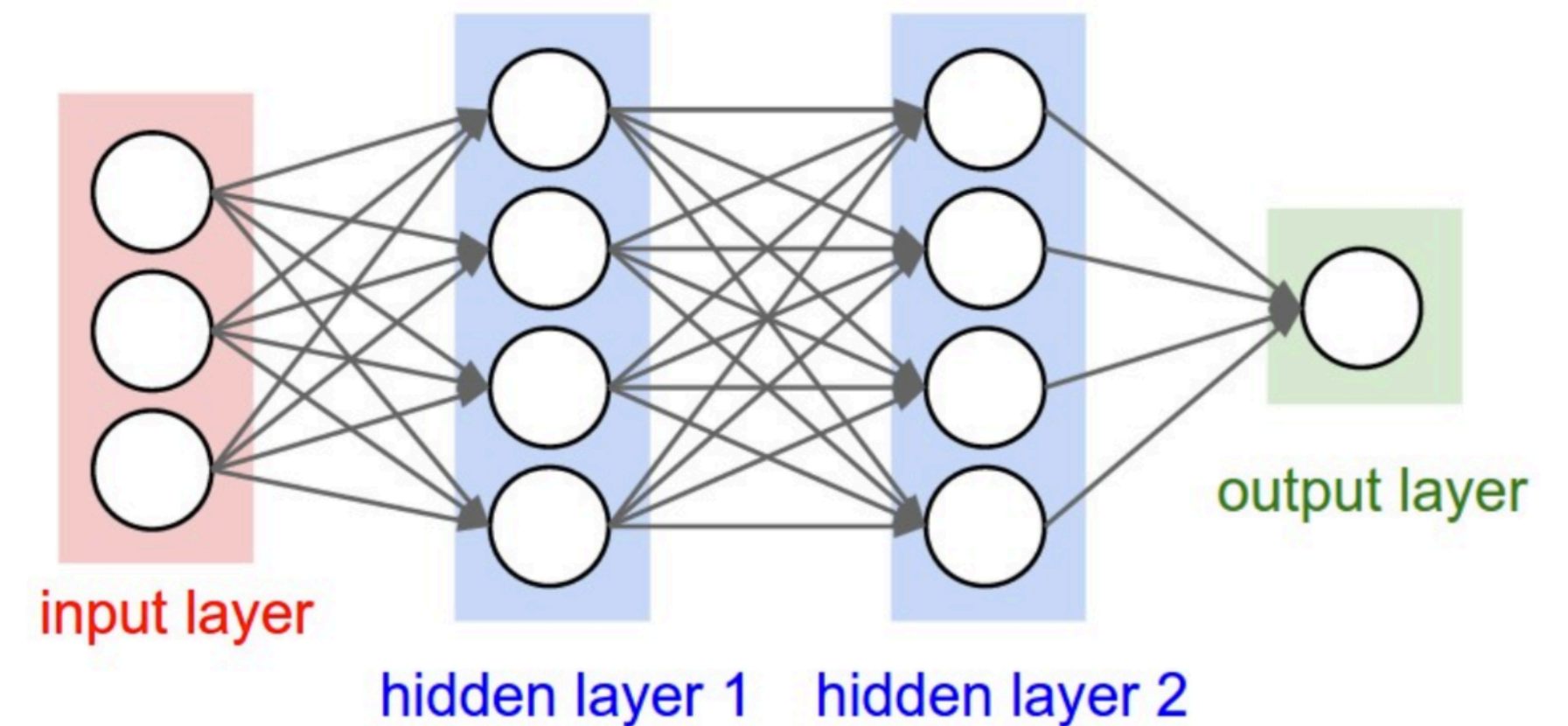
- Hidden layer 2:

$$\mathbf{h}_2 = f(\mathbf{W}^{(2)}\mathbf{h}_1 + \mathbf{b}^{(2)}) \in \mathbb{R}^{d_2}$$

$$\mathbf{W}^{(2)} \in \mathbb{R}^{d_2 \times d_1}, \mathbf{b}^{(2)} \in \mathbb{R}^{d_2}$$

- Output layer:

$$y = \sigma(\mathbf{w}^{(o)} \cdot \mathbf{h}_2 + b^{(o)})$$

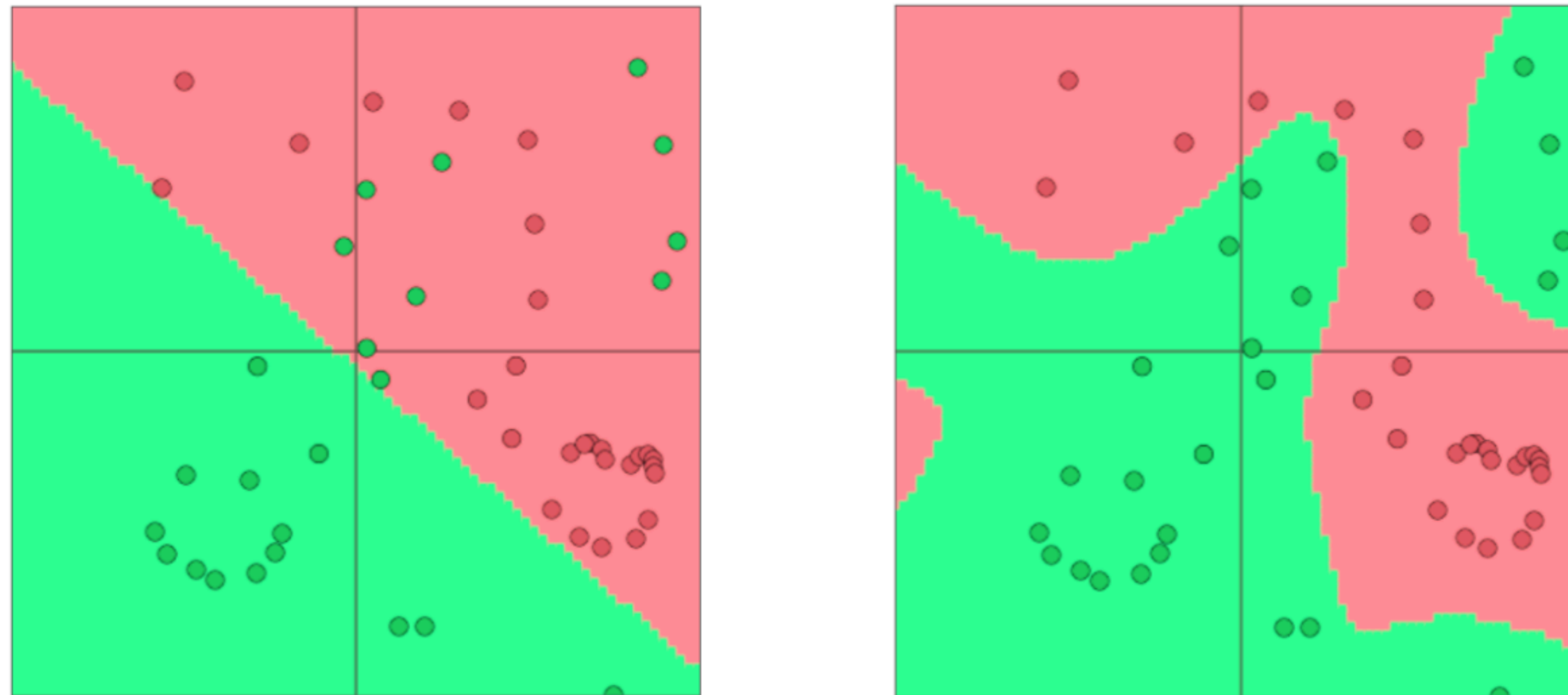


Note: f is applied element-wise

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)]$$

Why non-linearities?

- Neural networks can learn much more complex functions and nonlinear decision boundaries



The capacity of the network increases with more hidden units and more hidden layers

What if we remove activation function?

XOR problem

We will assume a training set where each label is in the set $\mathcal{Y} = \{-1, +1\}$

There are four training examples:

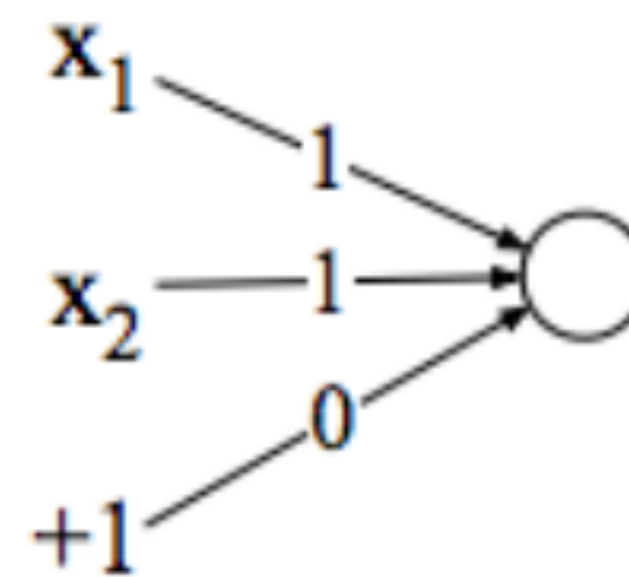
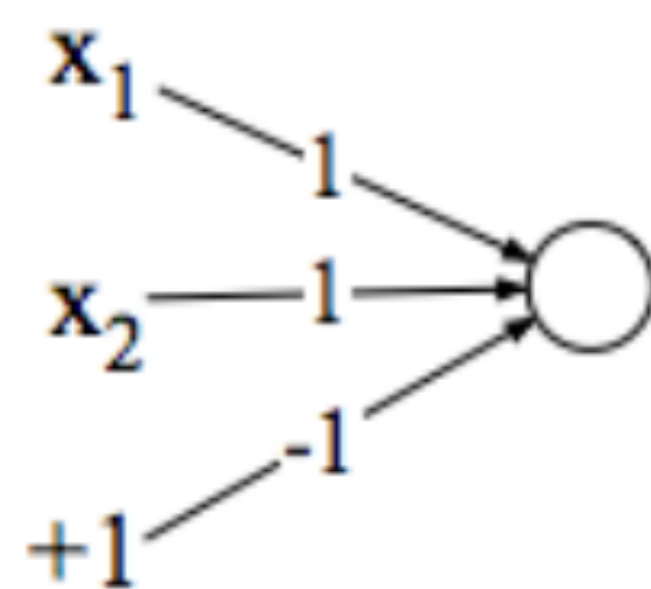
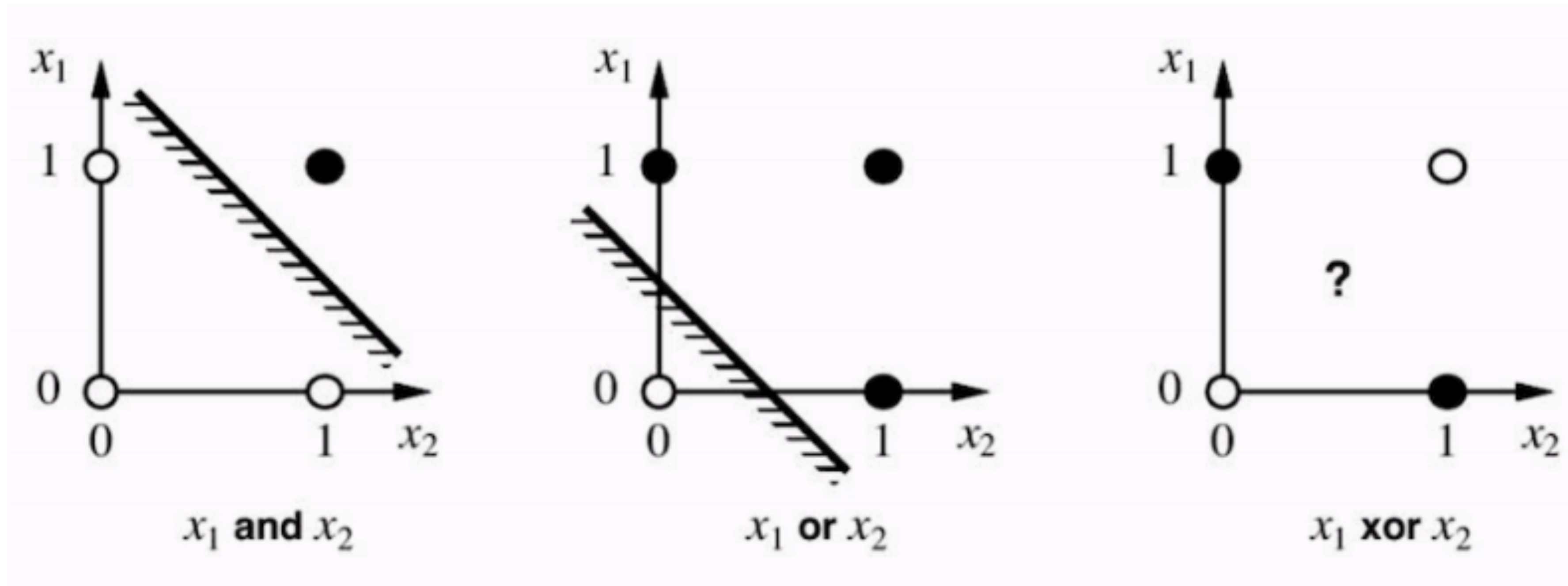
$$x^1 = [0, 0], y^1 = -1$$

$$x^2 = [0, 1], y^2 = +1$$

$$x^3 = [1, 0], y^3 = +1$$

$$x^4 = [1, 1], y^4 = -1$$

Single neuron (perceptron)



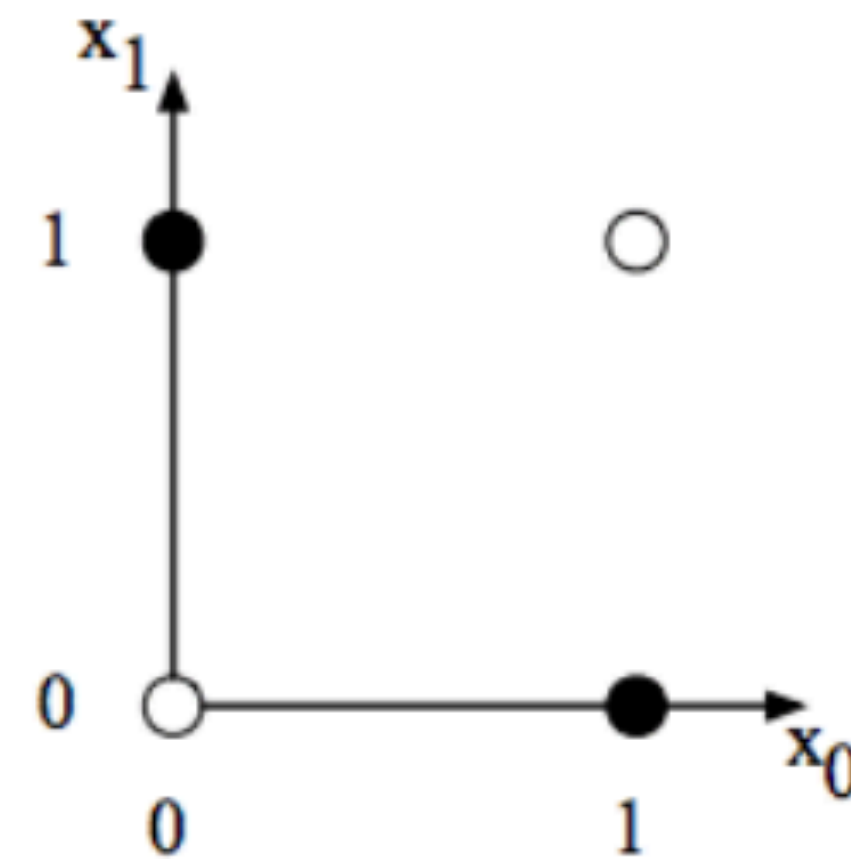
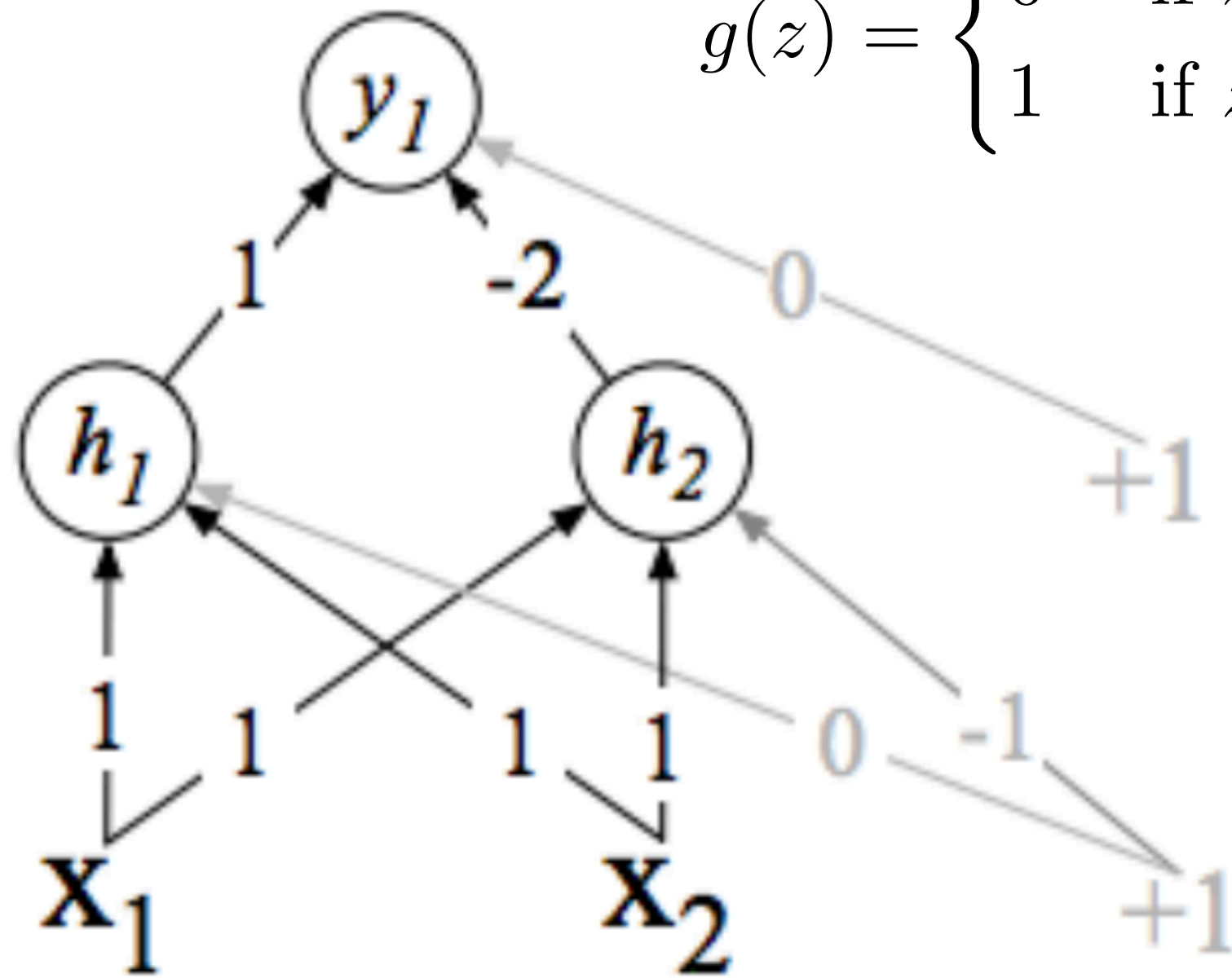
Perceptron can compute **and** and **or**

$$y = \begin{cases} 0 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq 0 \\ 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0 \end{cases}$$

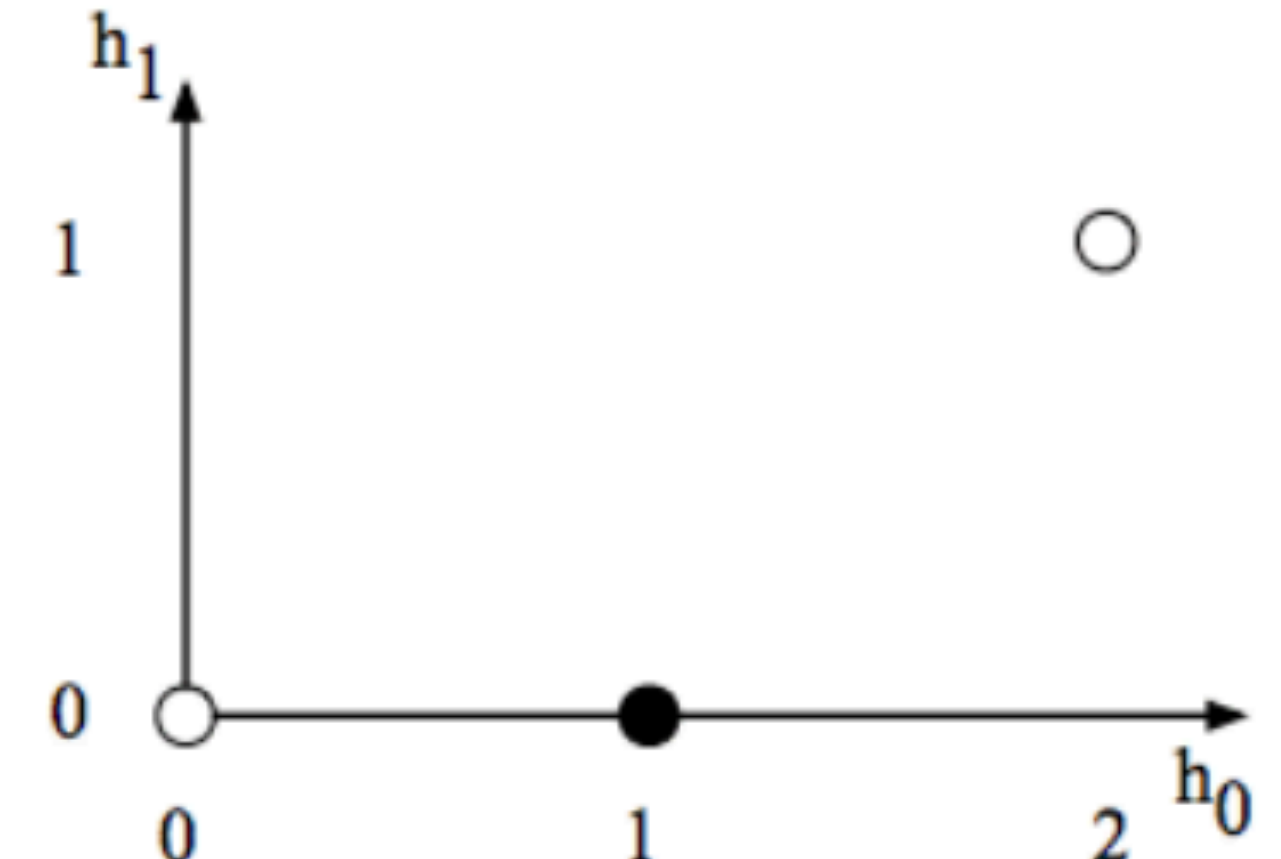
Solution: multiple neurons

$$y_1 = g(h_1 - 2h_2)$$

$$g(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$



a) The original x space



b) The new h space

$$h_1 = g(x_1 + x_2)$$

$$h_2 = g(x_1 + x_2 - 1)$$